Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 4

Mrežna konfiguracija i firewall kao kod

Mreža kao kod: osnovni mrežni zadaci naredbom `ip` (adrese, sučelja, rute), segmenti i usmjeravanje (`VPC/subnet`, routing, NAT) te vatrozid u `nftables`. Konfiguracija se opisuje datotekama i podacima, a ishod se provjerava testom povezivosti.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 06.11.2026, 17:15–20:15, Kabinet 3204
TEMA	Mrežna konfiguracija i firewall kao kod
KOD	github.com/Dadyze/RuO-2026-27 (mape `s4/p1` do `s4/p4`)

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

U prethodne tri sedmice su mašine i servisi opisani kodom. Ostaje pitanje *ko s kim smije razgovarati*. Mreža i vatrozid su najčešći izvor grešaka i sigurnosnih propusta, a ručno podešavanje je teško ponoviti i još teže pregledati. Zato se i oni pišu kao kod: pravila su u datoteci, datoteka je u Gitu, a ishod se provjerava testom, ne pogađanjem.

Za rad nije potreban račun u oblaku. Mrežni elementi (ruter, segmenti) prave se od **Linux network namespace-ova**, istog mehanizma na kojem se zasnivaju kontejneri iz prve sedmice. Jedan namespace je jedna izolirana mreža sa svojim sučeljima, adresama i tabelom usmjeravanja, pa tako radi i svaka mašina u labu.

Nakon nastavne jedinice student je u stanju:

- pregledati, dodati, promijeniti i obrisati IP adresu, sučelje i rutu naredbom `ip` te objasniti zašto su te promjene privremene,
- povezati pojmove oblaka (VPC, subnet, route table, NAT, security group) s Linux alatima koji ih ostvaruju,
- napraviti segmentiranu mrežu s ruterom i NAT-om i objasniti put paketa kroz nju,
- napisati stateful vatrozid s politikom "sve zabranjeno, dozvole se nabrajaju" u `nftables` i provjeriti ga matricom povezivosti,

- generisati pravila vatrozida iz podataka Ansible rolom i provjeriti idempotentnost,

Okruženje i pokretanje

Alate za ovu sedmicu (`ip`, `ping` i `nft`) instalira `00-setup.sh`. Ako je već pokrenuta ranije, ponovo se pokrene (idempotentna je), pa se primjeri pokreću redom:

```bash
cd ~/RuO-2026-27 && git pull
./00-setup.sh                    # dodaje iproute2, iputils-ping i nftables
make check                       # da li su nft i ip prisutni
make s4-p1                       # pregled mreže; zatim s4-p1-lab, s4-p2 ... s4-p4; opcionalni
koraci imaju sufiks -lab, -up ili -apply
```

RAD BEZ TROŠKOVA: PREGLED UMJESTO IZVRŠAVANJA

Osnovni cilj svakog primjera samo pokazuje šta bi se desilo: `DRY=1` ispisuje naredbe bez izvršavanja, `nft -c` provjerava sintaksu bez učitavanja, a `ansible-playbook --check --diff` prikazuje razliku. Stvarno izvršavanje ima sufiks `-lab`, `-up` ili `-apply`, traži `sudo` i mijenja samo mrežu u izoliranim namespace-ovima, ne mrežu računara. Sve se briše sa `make s4-clean`.

VAŽNO

Primjeri `-lab`, `-up` i `-apply` kreiraju namespace-ove i učitavaju pravila, pa traže administratorska prava. Primjeri `s4-p2` i dalje se nastavljaju jedan na drugi: prvo `s4-p2-up` (lab), zatim `s4-p3-apply` (vatrozid). Vježba iz `s4-p1` je samostalna.

---

1 · POJAM I POZADINA

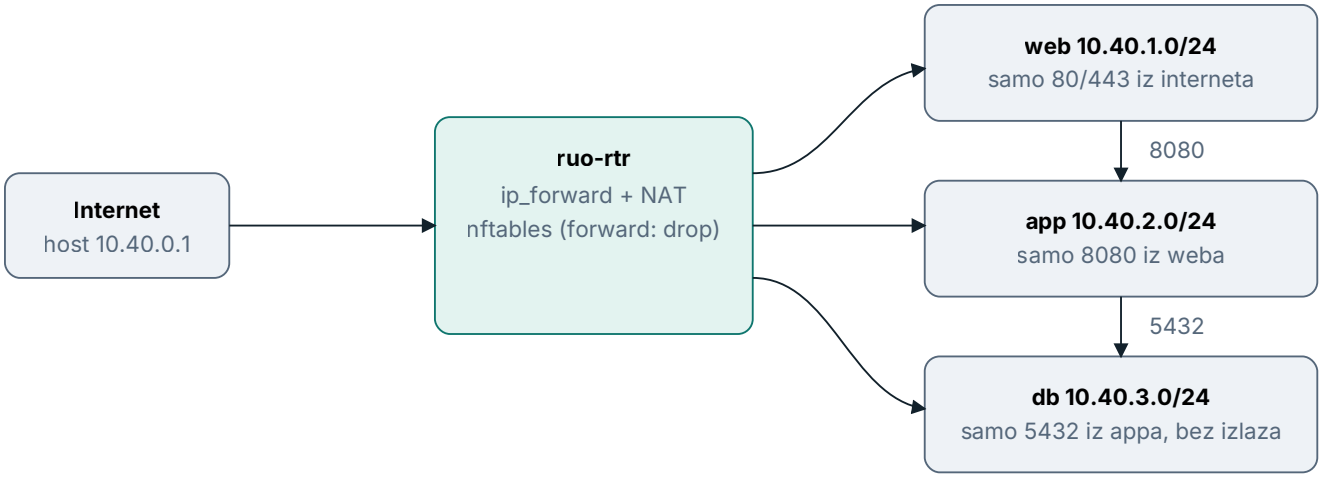
Od mašina do mreže

Mreža u oblaku se sastoji od nekoliko elemenata koje svaki pružatelj (AWS, Azure, GCP) naziva drugačije, ali rade isto. Na Linuxu svaki od njih ima konkretnu zamjenu, pa se isto razmišljanje može vježbati lokalno i besplatno.

OBLAK (AWS / AZURE)	ZNAČENJE	U OVOM LABU
VPC / VNet	izolirana privatna mreža	ruter `ruo-rtr` i namespace-ovi iza njega
Subnet	podmreža unutar VPC-a	segment `10.40.N.0/24` (namespace + veth par)
Route table	kuda ide koji promet	`ip route`, zadani put preko rutera
Internet / NAT gateway	izlaz privatnih mašina prema internetu	uplink `r-up` + `masquerade`
Security group / NSG	stateful pravila uz mašinu ili podmrežu	`nftables` lanac `forward`, zadano `drop`

POZADINA: NFTABLES

Linux vatrozid je dugo bio `iptables` (više odvojenih alata za IPv4, IPv6, ARP). `nftables` ga je zamijenio 2014. godine (jezgra 3.13): jedan alat, jedan jezik, skupovi (*sets*) i atomsko učitavanje cijele datoteke, a od Debiana 10 je i zadani. Pravila su organizovana u **tabele** (`table`), tabele u **lance** (`chain`), a lanci se vežu na *hook* u putu paketa kroz jezgru: `input` (paket za sam ruter), `forward` (paket koji prolazi kroz ruter), `output` i `prerouting` / `postrouting` (NAT). Praćenje veza (*connection tracking*, `ct`) pamti stanje, pa vatrozid dozvoljava odgovore na dozvoljene veze bez posebnog pravila.



Slika 1. Topologija laba: sav promet između segmenata prolazi kroz ruter, pa se politika provodi na jednom mjestu. Strelice između segmenata su jedini dozvoljeni putevi.

ZAŠTO SEGMENTACIJA

Ako napadač preuzme web server, u ravnoj mreži odmah vidi i bazu. Segmentacijom je web smije dosegnuti samo kroz aplikacijski port, a bazu uopće ne. Politika "sve zabranjeno, dozvole se nabrajaju" (*default deny*) znači da zaboravljeno pravilo uzrokuje prekid veze koji se odmah primijeti, a ne tihu rupu.

---

2 · PRIMJER 1 · S4/P1

Osnovni mrežni zadaci: pregled, dodavanje, mijenjanje i brisanje

POZADINA PRIMJERA

Naredba `ip` (paket *iproute2*) zamijenila je stare `ifconfig` i `route`. Radi s tri vrste objekata: `ip link` (**sučelje**: kartica ili virtuelni kabl, može biti uključeno ili isključeno), `ip addr` (**adresa**: IP uz prefiks, npr. `10.60.0.1/24` znači adresu `10.60.0.1` u mreži od 24 bita) i `ip route` (**ruta**: kroz koje sučelje, i preko kojeg *gatewaya*, ide promet za neku mrežu). Mašina može slati IP promet tek kad sučelje ima adresu i uključeno je; jezgra pri dodavanju adrese sama dodaje rutu za njenu podmrežu. Naredbe mijenjaju *samo trenutno stanje*: restart ih briše, pa se trajna konfiguracija piše datotekom (netplan, vidi niže).

ZADATAK	NAREDBA
pregled sučelja / adresa / ruta	`ip -br link` · `ip -br addr` · `ip route`
uključi / isključi sučelje	`ip link set eth0 up` · `ip link set eth0 down`
dodaj adresu	`ip addr add 10.60.0.1/24 dev eth0`
promijeni adresu	`ip addr del` stare, pa `ip addr add` nove (ili `ip addr replace`)
obriši adresu / sve adrese	`ip addr del 10.60.0.11/24 dev eth0` · `ip addr flush dev eth0`
promijeni MTU	`ip link set eth0 mtu 1400`
dodaj / obriši rutu	`ip route add 10.70.0.0/24 via 10.60.0.22` · `ip route del 10.70.0.0/24`
kojim putem ide paket	`ip route get 10.70.0.1`
brojači paketa i grešaka	`ip -s link show eth0`

bash — Isječak 1 · pokretanje

```
make s4-p1           # pregled mreže računara (samo čita)
make s4-p1-lab       # opcionalno: cijela vježba u dva namespace-a (traži sudo); PAUZA=1 čeka Enter
make s4-p1-clean
```

Vježba (`s4/p1/osnove.sh`) pravi dva namespace-a, `ruo-n1` i `ruo-n2`, spojena veth kablom, pa mreža računara ostaje netaknuta. Svaki korak ispiše naredbu, izvrši je i pokaže stanje; `ping` pokazuje da li promjena radi. Prvi dio: sučelje je isključeno i bez adrese, zatim se uključuje, dobija adresu i druga adresa.

izlaz make s4-p1-lab: koraci 1 do 4 (skraćeno)

```
sudo PAUZA= bash s4/p1/osnove.sh vjezba

== 1. Početno stanje: sučelje postoji, ali je isključeno i nema adresu
[ruo-n1]$ ip -br link
lo               DOWN           00:00:00:00:00:00 <LOOPBACK>
eth0@if2         DOWN           06:26:11:47:60:d6 <BROADCAST,MULTICAST>
[ruo-n1]$ ip -br addr
lo               DOWN
eth0@if2         DOWN

== 2. Uključivanje sučelja (up): bez adrese ono još ne može slati IP promet
[ruo-n1]$ ip link set lo up
[ruo-n1]$ ip link set eth0 up
[ruo-n2]$ ip link set lo up
[ruo-n2]$ ip link set eth0 up
[ruo-n1]$ ip -br addr
lo               UNKNOWN        127.0.0.1/8
eth0@if2         UP

== 3. Dodavanje adrese (ip addr add): oblik adresa/prefiks; jezgra sama dodaje rutu za podmrežu
[ruo-n1]$ ip addr add 10.60.0.1/24 dev eth0
[ruo-n2]$ ip addr add 10.60.0.2/24 dev eth0
[ruo-n1]$ ip -br addr
lo               UNKNOWN        127.0.0.1/8
eth0@if2         UP             10.60.0.1/24
[ruo-n1]$ ip route
10.60.0.0/24 dev eth0 proto kernel scope link src 10.60.0.1
[ruo-n1] ping 10.60.0.2: ODGOVARA
```

Adresa se ne "uređuje": mijenjanje je brisanje stare i dodavanje nove, što se vidi po tome da stara adresa prestaje odgovarati na ping. Isto vrijedi za sučelje: isključeno sučelje prekida promet iako adresa ostaje.

izlaz: koraci 5 i 6 (mijenjanje adrese i svojstava sučelja)

```
== 4. Druga adresa na istom sučelju (dozvoljeno: sučelje može imati više adresa)
[ruo-n1]$ ip addr add 10.60.0.11/24 dev eth0
[ruo-n1]$ ip -br addr
lo               UNKNOWN        127.0.0.1/8
eth0@if2         UP             10.60.0.1/24 10.60.0.11/24

== 5. Mijenjanje adrese: IP se ne 'uređuje', stara se briše a nova dodaje (n2: .2 -> .22)
[ruo-n2]$ ip addr del 10.60.0.2/24 dev eth0
[ruo-n2]$ ip addr add 10.60.0.22/24 dev eth0
[ruo-n1] ping 10.60.0.2: NE ODGOVARA
[ruo-n1] ping 10.60.0.22: ODGOVARA
```

GREŠKE KOJE SE NAJČEŠĆE PRAVE

Dodavanje adrese bez prefiksa (`10.60.0.1` bez `/24`) daje prefiks `/32`, pa nema rute za podmrežu i ništa osim same adrese ne radi. Zaboravljeno `ip link set ... up` ostavlja sučelje isključenim (`state DOWN`). Dvije različite mašine s istom adresom prave prekide koji se teško traže, pa se prije dodavanja provjerava postoji li adresa (`ip -br addr`).

Ruta je potrebna kad je cilj izvan direktno spojenih mreža. U vježbi `ruo-n2` ima mrežu `10.70.0.0/24` koju `ruo-n1` ne poznaje: ping ne prolazi dok se ne doda ruta preko `ruo-n2`, a prestaje čim se obriše. `ip route get` pokazuje koju će rutu jezgra izabrati za dati cilj.

izlaz: koraci 7 i 8 (rute i brisanje adresa)

```
== 6. Mijenjanje svojstava sučelja: MTU (najveća veličina paketa) i isključivanje/uključivanje
[ruo-n1]$ ip link set eth0 mtu 1400
[ruo-n1]$ ip link show eth0
2: eth0@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc noqueue state UP mode DEFAULT
group default qlen 1000
    link/ether 06:26:11:47:60:d6 brd ff:ff:ff:ff:ff:ff link-netns ruo-n2
[ruo-n1]$ ip link set eth0 down
[ruo-n1] ping 10.60.0.22: NE ODGOVARA
[ruo-n1]$ ip link set eth0 up
[ruo-n1] ping 10.60.0.22: ODGOVARA

== 7. Rute: n2 ima još jednu mrežu (10.70.0.1 na lo); n1 do nje nema puta dok se ruta ne doda
[ruo-n2]$ ip addr add 10.70.0.1/24 dev lo
[ruo-n1] ping 10.70.0.1: NE ODGOVARA
[ruo-n1]$ ip route add 10.70.0.0/24 via 10.60.0.22
[ruo-n1]$ ip route get 10.70.0.1
10.70.0.1 via 10.60.0.22 dev eth0 src 10.60.0.1 uid 0
    cache
[ruo-n1] ping 10.70.0.1: ODGOVARA
[ruo-n1]$ ip route del 10.70.0.0/24
[ruo-n1] ping 10.70.0.1: NE ODGOVARA
```

izlaz: korak 9 (statistika sučelja)

```
== 8. Brisanje adrese (ip addr del) i brisanje svih adresa sa sučelja (ip addr flush)
[ruo-n1]$ ip addr del 10.60.0.11/24 dev eth0
[ruo-n1]$ ip -br addr
lo               UNKNOWN        127.0.0.1/8
eth0@if2         UP             10.60.0.1/24
[ruo-n1]$ ip addr flush dev eth0
[ruo-n1]$ ip -br addr
lo               UNKNOWN        127.0.0.1/8
eth0@if2         UP
[ruo-n1]$ ip route
(prazna tabela usmjeravanja: rute iz adrese nestaju zajedno s njom)
[ruo-n1] ping 10.60.0.22: NE ODGOVARA
```

Trajna konfiguracija: netplan

Sve gore je nestalo restartom. Na Ubuntuu se stalna mrežna konfiguracija opisuje datotekom u `/etc/netplan/` (YAML, isti princip kao ostatak kursa: željeno stanje u datoteci, ne niz ručnih naredbi). Primjer opisuje isto što i naredbe `ip addr add` i `ip route add` iz vježbe:

yaml — Isječak 2 · s4/p1/netplan-primjer.yaml

```yaml
# Trajna konfiguracija mreže na Ubuntuu (netplan). Naredbe ip mijenjaju stanje samo do
restarta;
# ova datoteka opisuje isto što i ip addr add / ip route add, ali kao kod koji preživi restart.
# Primjena: sudo cp netplan-primjer.yaml /etc/netplan/60-ruo.yaml && sudo netplan try
network:
  version: 2
  ethernets:
    eth0:
      addresses:
        - 10.60.0.1/24          # ip addr add 10.60.0.1/24 dev eth0
      routes:
        - to: 10.70.0.0/24      # ip route add 10.70.0.0/24 via 10.60.0.22
          via: 10.60.0.22
```

PROVJERA I PRIMJENA

Prije primjene se datoteka provjerava, a `sudo netplan try` primjenjuje promjenu uz automatski povrat ako se u roku od nekoliko sekundi ne potvrdi, što štiti od gubitka veze pri radu na udaljenoj mašini. Naredba `netplan generate` pokazuje šta će netplan proslijediti sistemu (`systemd-networkd` ili `NetworkManager`). Primjer je provjeren tom naredbom. Na WSL2 mrežom upravlja Windows, pa se netplan tamo ne koristi.

---

3 · PRIMJER 2 · S4/P2

Segmenti, ruter i NAT od namespace-ova

POZADINA PRIMJERA

Svaki segment je namespace (`ruo-web`, `ruo-app`, `ruo-db`) spojen na ruter `ruo-rtr` **veth parom**: virtualnim kablom čiji se jedan kraj nalazi u segmentu, a drugi u ruteru. Ruter ima po jedno sučelje u svakom segmentu (`.1` u podmreži) i uplink prema "internetu" (host). Mašine u segmentu imaju zadani put (`default via`) preko rutera. Bez `net.ipv4.ip_forward=1` ruter ne bi prosljeđivao pakete, što je čest razlog zašto "sve je spojeno, a ništa ne radi".

bash — Isječak 3 · s4/p2/lab.sh (izvod)

```bash
  # 1. ruter i uplink prema "internetu" (host)
  run ip netns add ruo-rtr
  run ip link add h-up type veth peer name r-up netns ruo-rtr
  run ip addr add 10.40.0.1/30 dev h-up
  run ip link set h-up up
  run ip -n ruo-rtr addr add 10.40.0.2/30 dev r-up
  run ip -n ruo-rtr link set lo up
  run ip -n ruo-rtr link set r-up up
  run ip route add 10.40.0.0/16 via 10.40.0.2          # povratni put do segmenata
  rtr sysctl -qw net.ipv4.ip_forward=1                 # ruter prosljeđuje pakete
  run ip -n ruo-rtr route add default via 10.40.0.1

  # 2. segmenti: namespace + veth par prema ruteru + zadani put preko rutera
  for s in $SEGMENTI; do
    ime="${s%%:*}"; n="${s##*:}"
    run ip netns add "ruo-$ime"
    run ip link add "r-$ime" netns ruo-rtr type veth peer name eth0 netns "ruo-$ime"
```

```
    run ip -n ruo-rtr addr add "10.40.$n.1/24" dev "r-$ime"
    run ip -n ruo-rtr link set "r-$ime" up
    run ip -n "ruo-$ime" addr add "10.40.$n.2/24" dev eth0
    run ip -n "ruo-$ime" link set lo up
    run ip -n "ruo-$ime" link set eth0 up
    run ip -n "ruo-$ime" route add default via "10.40.$n.1"
  done
```

bash — Isječak 4 · pokretanje

```
make s4-p2          # DRY=1: ispisuje naredbe, ništa ne mijenja
make s4-p2-up       # opcionalno: stvarno pravi ruter i tri segmenta (traži sudo)
```

izlaz make s4-p2-up: stanje rutera (skraćeno, imena sučelja se mogu razlikovati)

```
== ruo-rtr
r-up@if6         UP             10.40.0.2/30
r-web@if2        UP             10.40.1.1/24
r-app@if2        UP             10.40.2.1/24
r-db@if2         UP             10.40.3.1/24
default via 10.40.0.1 dev r-up
10.40.0.0/30 dev r-up proto kernel scope link src 10.40.0.2
10.40.1.0/24 dev r-web proto kernel scope link src 10.40.1.1
10.40.2.0/24 dev r-app proto kernel scope link src 10.40.2.1
10.40.3.0/24 dev r-db proto kernel scope link src 10.40.3.1
```

Ruter ima put do svake podmreže jer su mu sučelja direktno u njima (`proto kernel scope link`), a sve ostalo ide na `default via 10.40.0.1`. Za povratni put host mora znati da je `10.40.0.0/16` iza rutera; to je jedan red u skripti (`ip route add 10.40.0.0/16 via 10.40.0.2`). Isti princip vrijedi u oblaku: odgovor mora imati put nazad, a u tabeli usmjeravanja je to *route table* podmreže.

> **NAT**
> Privatne adrese (`10.0.0.0/8`) se ne usmjeravaju po internetu. **Masquerade** mijenja izvornu adresu paketa u adresu izlaznog sučelja rutera i pamti prevod, pa se odgovor vraća pravoj mašini. To je *SNAT* kakav radi kućni ruter i NAT gateway u oblaku. Pravilo je u lancu `postrouting` (Primjer 3).

4 · PRIMJER 3 · S4/P3

Vatrozid u nftables: default deny i segmentacija

> **POZADINA PRIMJERA**
> Pravila se učitavaju u ruter, pa vrijede za sav promet između segmenata. Lanac `forward` ima `policy drop`: sve što nijedno pravilo ne dozvoljava pada. Prvo pravilo (`ct state established,related accept`) propušta odgovore na dozvoljene veze, pa je dovoljno opisati samo *smjer uspostave* veze (stateful vatrozid, kao security group u oblaku). Datoteka počinje s `table inet ruo` i `delete table inet ruo`: prvi red osigurava da tabela postoji (inače bi brisanje javilo grešku), a drugi je briše, pa učitavanje uvijek počinje od čistog stanja i može se ponavljati (**idempotentnost**). Cijela datoteka se učitava atomski: ili sva pravila, ili nijedno.

nft — Isječak 5 · s4/p3/firewall.nft

```
#!/usr/sbin/nft -f
# s4/p3: segmentacija web/app/db na ruteru (ruo-rtr). Učitava se atomski: cijela datoteka ili ništa.
# Idiom "table ...; delete table ...; table ... { }" čini učitavanje idempotentnim bez diranja tuđih tabela.

define web    = 10.40.1.0/24
define app    = 10.40.2.0/24
define db     = 10.40.3.0/24
define uplink = "r-up"

table inet ruo
delete table inet ruo

table inet ruo {
  set web_ports { type inet_service; elements = { 80, 443 } }

  # promet KROZ ruter (između segmenata): zadano je zabrana, dozvole se nabrajaju
  chain forward {
    type filter hook forward priority filter; policy drop;

    ct state established,related accept          # odgovori na dozvoljene veze
    ct state invalid drop

    iifname $uplink ip daddr $web tcp dport @web_ports accept   # internet -> web: samo 80/443
    ip saddr $web ip daddr $app tcp dport 8080 accept
# web -> app: samo aplikacijski port
    ip saddr $app ip daddr $db  tcp dport 5432 accept           # app -> db: samo baza
    ip saddr { $web, $app } oifname $uplink accept              # izlaz prema internetu: web i app, ne db

    limit rate 5/minute log prefix "ruo-drop: "                 # sve ostalo pada (policy drop)
  }

  # promet KA samom ruteru
  chain input {
    type filter hook input priority filter; policy drop;
    iifname "lo" accept
    ct state established,related accept
```

```
    icmp type echo-request limit rate 5/second accept
  }

  # NAT: segmenti izlaze prema uplinku s adresom rutera
  chain postrouting {
    type nat hook postrouting priority srcnat;
    oifname $uplink ip saddr 10.40.0.0/16 counter masquerade
  }
}
```

POJAM	ZNAČENJE
`table inet ruo`	kontejner za lance; `inet` pokriva IPv4 i IPv6 jednim skupom pravila
`chain ... hook forward`	lanac vezan na tačku u jezgri; `priority` određuje redoslijed među lancima na istom hooku
`policy drop`	zadana odluka kad nijedno pravilo ne pogodi paket
`ct state`	stanje veze iz connection trackinga (`established`, `related`, `invalid`)
`set web_ports`	imenovani skup (portova, adresa); mijenja se na jednom mjestu
`define`	varijabla na nivou datoteke, razrješava se pri učitavanju
`masquerade`	NAT koji koristi adresu izlaznog sučelja (lanac `postrouting`)

> **GRANICA: VIŠE TABELA**
> Paket mora proći *sve* lance na istom hooku. `accept` u jednoj tabeli završava samo taj lanac, pa druga tabela s `policy drop` i dalje može odbaciti isti paket. Zato se dozvola dodaje u tabelu koja zabranjuje, a ne u zasebnu.

bash — Isječak 6 · pokretanje

```
make s4-p3          # nft -c: provjera sintakse bez učitavanja
make s4-p3-apply    # opcionalno: matrica PRIJE pa POSLIJE učitavanja (traži s4-p2-up)
```

Matrica povezivosti (`s4/p3/test.sh`) na svako odredište postavi TCP slušaoca, pa pokuša vezu iz zadanog izvora. Prva kolona je izvor, zatim odredište i port, ishod i očekivanje iz politike. Prije vatrozida je mreža ravna i sve prolazi (šest razlika od politike), a poslije je ishod jednak politici u svih deset slučajeva:

izlaz PRIJE učitavanja firewall.nft

```
  host   -> 10.40.1.2  :80     dozvoljeno  (očekivano: dozvoljeno) ok
  host   -> 10.40.2.2  :8080   dozvoljeno  (očekivano: blokirano ) RAZLIKA
  host   -> 10.40.3.2  :5432   dozvoljeno  (očekivano: blokirano ) RAZLIKA
  web    -> 10.40.2.2  :8080   dozvoljeno  (očekivano: dozvoljeno) ok
  web    -> 10.40.2.2  :22     dozvoljeno  (očekivano: blokirano ) RAZLIKA
  web    -> 10.40.3.2  :5432   dozvoljeno  (očekivano: blokirano ) RAZLIKA
  app    -> 10.40.3.2  :5432   dozvoljeno  (očekivano: dozvoljeno) ok
  app    -> 10.40.3.2  :22     dozvoljeno  (očekivano: blokirano ) RAZLIKA
  web    -> 10.40.0.1  :9000   dozvoljeno  (očekivano: dozvoljeno) ok
  db     -> 10.40.0.1  :9000   dozvoljeno  (očekivano: blokirano ) RAZLIKA
```

izlaz POSLIJE učitavanja firewall.nft

```
  host   -> 10.40.1.2  :80     dozvoljeno  (očekivano: dozvoljeno) ok
  host   -> 10.40.2.2  :8080   blokirano   (očekivano: blokirano ) ok
  host   -> 10.40.3.2  :5432   blokirano   (očekivano: blokirano ) ok
  web    -> 10.40.2.2  :8080   dozvoljeno  (očekivano: dozvoljeno) ok
  web    -> 10.40.2.2  :22     blokirano   (očekivano: blokirano ) ok
  web    -> 10.40.3.2  :5432   blokirano   (očekivano: blokirano ) ok
  app    -> 10.40.3.2  :5432   dozvoljeno  (očekivano: dozvoljeno) ok
  app    -> 10.40.3.2  :22     blokirano   (očekivano: blokirano ) ok
  web    -> 10.40.0.1  :9000   dozvoljeno  (očekivano: dozvoljeno) ok
  db     -> 10.40.0.1  :9000   blokirano   (očekivano: blokirano ) ok
```

nft — Isječak 7 · nft list chain inet ruo forward (stanje u jezgri)

```
table inet ruo {
  chain forward {
    type filter hook forward priority filter; policy drop;
    ct state established,related accept
    ct state invalid drop
    iifname "r-up" ip daddr 10.40.1.0/24 tcp dport @web_ports accept
    ip saddr 10.40.1.0/24 ip daddr 10.40.2.0/24 tcp dport 8080 accept
    ip saddr 10.40.2.0/24 ip daddr 10.40.3.0/24 tcp dport 5432 accept
    ip saddr { 10.40.1.0-10.40.2.255 } oifname "r-up" accept
    limit rate 5/minute burst 5 packets log prefix "ruo-drop: "
  }
}
```

ŠTA JEZGRA RADI S PRAVILIMA

Ispis stanja se razlikuje od izvorne datoteke: `@web_ports` ostaje skup, a lista `{ $web, $app }` je spojena u interval `10.40.1.0-10.40.2.255` jer su podmreže susjedne. Smisao je isti, ali treba znati čitati ispis. Brojači (`counter`) se mogu dodati pravilima da se vidi koliko je paketa pogodilo koje pravilo, što je prvi alat za otkrivanje zašto veza ne radi.

---

5 · PRIMJER 4 · S4/P4

Vatrozid iz podataka: Ansible rola

POZADINA PRIMJERA

Ručno pisanje pravila ne skalira: nova usluga znači izmjenu više redova na više mjesta. Ovdje se politika opisuje **podacima** (koji segment smije kojem, na kojem portu), a šablon ih pretvara u `nftables`. Tako se pravila pregledaju kao podaci, a generisanje je jedno mjesto koje se testira. Rola koristi alate iz sedmice 3: šablon (Jinja2), handler i `validate`, kojim se izgenerisana datoteka provjerava *prije* nego što zamijeni radnu.

yaml — Isječak 8 · s4/p4/group_vars/rtr/vars.yml

```
# Politika mreže kao PODACI: pravila se mijenjaju ovdje, a šablon ih pretvara u nftables.
fw_uplink: r-up

fw_segmenti:                    # ime: podmreža
  web: 10.40.1.0/24
```

```
  app: 10.40.2.0/24
  db: 10.40.3.0/24

fw_javno:                       # internet -> segment
  - { do: web, portovi: [80, 443] }

fw_pravila:                     # segment -> segment (slojevi: web -> app -> db)
  - { od: web, do: app, portovi: [8080] }
  - { od: app, do: db,  portovi: [5432] }

fw_izlaz: [web, app]            # segmenti kojima je dozvoljen izlaz prema internetu (db nije)

# U labu se pravila učitavaju u namespace rutera iz s4/p2; na pravom ruteru ostaje samo "nft".
fw_nft_cmd: ip netns exec ruo-rtr nft
fw_dest: /etc/nftables.d/ruo.nft
fw_instaliraj_paket: true
```

jinja2 — Isječak 9 · s4/p4/roles/firewall/templates/ruo.nft.j2 (izvod)

```
  chain forward {
    type filter hook forward priority filter; policy drop;

    ct state established,related accept
    ct state invalid drop

{% for r in fw_javno %}
    # internet -> {{ r.do }}
    iifname "{{ fw_uplink }}" ip daddr {{ fw_segmenti[r.do] }} tcp dport { {{ r.portovi |
join(', ') }} } accept
{% endfor %}
{% for r in fw_pravila %}
    # {{ r.od }} -> {{ r.do }}
    ip saddr {{ fw_segmenti[r.od] }} ip daddr {{ fw_segmenti[r.do] }} tcp dport { {{ r.portovi
| join(', ') }} } accept
{% endfor %}
{% for s in fw_izlaz %}
    # izlaz: {{ s }} -> internet
    ip saddr {{ fw_segmenti[s] }} oifname "{{ fw_uplink }}" accept
{% endfor %}
  }
```

yaml — Isječak 10 · s4/p4/roles/firewall/tasks/main.yml

```
- name: nftables je instaliran
  ansible.builtin.apt: { name: nftables, state: present, cache_valid_time: 3600 }
  when: fw_instaliraj_paket

- name: direktorij za pravila
  ansible.builtin.file: { path: "{{ fw_dest | dirname }}", state: directory, mode: "0755" }

- name: pravila iz šablona (provjera sintakse PRIJE zamjene datoteke)
  ansible.builtin.template:
    src: ruo.nft.j2
    dest: "{{ fw_dest }}"
    mode: "0644"
    validate: nft -c -f %s          # neispravna datoteka nikad ne zamijeni radnu
  notify: učitaj vatrozid
```

bash — Isječak 11 · pokretanje

```
make s4-p4           # syntax-check i --check --diff (bez izmjena)
make s4-p4-apply     # opcionalno: primjena dva puta, pa matrica povezivosti (traži s4-p2-up)
```

izlaz 1. izvršavanja (PLAY RECAP)

```
localhost : ok=4 changed=2 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

izlaz 2. izvršavanja: changed=0 = idempotentnost

```
localhost : ok=3 changed=0 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

Pravila koja je rola izgenerisala daju isti ishod kao ručno napisana datoteka iz Primjera 3: matrica povezivosti ponovo daje deset od deset. Politika se sada mijenja u `vars.yml`, a ne u pravilima.

SAŽETAK I PROVJERA ZNANJA

Pregled naredbi, pitanja i zadaci

CILJ	ŠTA RADI	MIJENJA SISTEM?
`make s4-p1` / `s4-p1-lab`	pregled mreže računara / vježba s adresama, sučeljima i rutama	ne / da (samo namespace-ovi, sudo)
`make s4-p2` / `s4-p2-up`	ispis naredbi (`DRY=1`) / ruter i segmenti web/app/db	ne / da (namespace-ovi, sudo)
`make s4-p3` / `s4-p3-apply`	`nft -c` / učitava vatrozid, matrica PRIJE/POSLIJE	ne / da (u ruteru laba)
`make s4-p4` / `s4-p4-apply`	rola vatrozida: `--check` / dva izvršavanja + matrica	ne / da
`make s4-clean`	briše sve iz sedmice 4	uklanja

PITANJE	ODGOVOR
Zašto se IP adresa "ne uređuje" nego briše i dodaje?	Adresa je zaseban objekt uz sučelje; `ip addr del` i `add` mijenjaju skup adresa, a rute izvedene iz adrese nestaju i nastaju s njom.
Šta se desi s promjenama napravljenim naredbom `ip` nakon restarta?	Nestaju, jer mijenjaju samo trenutno stanje jezgre. Trajna konfiguracija se piše datotekom (netplan).
Zašto `ping` prestaje raditi kad se sučelje isključi, a adresa ostane?	Isključeno sučelje ne šalje ni prima okvire; adresa je samo podatak uz sučelje.
Zašto ruter bez `ip_forward=1` ne prosljeđuje pakete?	Jezgra zadano odbacuje pakete koji nisu za nju; prosljeđivanje se mora uključiti, čime je ruter svjesna odluka, a ne nuspojava.
Čemu služi `ct state established,related accept`?	Propušta odgovore na već dozvoljene veze, pa se pravilima opisuje samo smjer uspostave veze.
Zašto datoteka počinje s `table` pa `delete table`?	Učitavanje počinje od čistog stanja i ne javlja grešku ako tabela ne postoji, pa je idempotentno.
Zašto jedna tabela s `accept` ne otvara ono što druga zabranjuje?	Paket mora proći sve lance na istom hooku; `accept` završava samo svoj lanac.
Čemu `validate` u zadatku `template`?	Neispravna datoteka se provjeri prije zamjene radne, pa greška u šablonu ne ruši vatrozid.

Zadaci za samostalni rad

1. U vježbi `osnove.sh` dodati treći namespace `ruo-n3` spojen na `ruo-n2` i podesiti rute tako da `ruo-n1` doseže `ruo-n3` preko `ruo-n2` (uz `ip_forward=1` na `ruo-n2`). Objasniti zašto bez forwardinga ping ne prolazi.
2. Napisati netplan datoteku za `ruo-n1` s dvije adrese i jednom rutom, pa je provjeriti naredbom `netplan generate`.

3. U `firewall.nft` dodati pravilo koje dozvoljava `web` prema `app` na port 9090, učitati ga i u `test.sh` dodati dvije provjere (dozvoljeno 9090, blokirano 9091). Objasniti zašto bez pravila veza ne radi.
4. Privremeno ukloniti `ct state established,related accept` i pokrenuti test. Objasniti šta se desi s odgovorima i zašto se pravilo vraća.
5. U `vars.yml` dodati segment `cache` (`10.40.4.0/24`) i pravilo `app` → `cache` na port 6379 te segment dodati u `SEGMENTI` u `lab.sh`. Pokrenuti rolu dva puta i predati izlaz drugog izvršavanja.
6. Svakom pravilu u lancu `forward` dodati `counter`, pokrenuti test i objasniti koji brojač je porastao pri kojoj provjeri.

> **SLJEDEĆA SEDMICA**
> Mreža i vatrozid su definisani kodom. Sedmica 5 uvodi imenovanje i komunikaciju: DNS zone i zapise (BIND/CoreDNS, Terraform DNS provideri), e-poštu (Postfix, Dovecot, SPF, DKIM, DMARC, TLS/ACME) te SFTP s chroot korisnicima iza vatrozida.
